

ScholarPath — Relational Mapping

From the EER model to relations (Elmasri & Navathe algorithm)

ScholarPath Graduation Project

June 30, 2026

Abstract

This report maps the conceptual EER model (see the companion *EERD* report) to a relational schema by applying the Elmasri & Navathe ER/EER-to-relational algorithm, then reconciles the result with the live Entity Framework Core model snapshot so that it matches the **actual deployed Azure SQL schema**. Every relation is listed with its primary key, foreign keys and delete rules.

Contents

1	Notation	1
2	How the EER constructs were mapped	2
3	Relational schema diagrams	2
4	Relation schemas (textual)	8
4.1	Identity, Access & Profile	8
4.2	Scholarships, Applications & Documents	9
4.3	Consultant Booking, Payments & Ratings	10
4.4	Community & Chat	11
4.5	Resources Hub & Notifications	12
4.6	AI, Knowledge, Platform & Cross-cutting	12
5	Referential-integrity summary	13
6	Exporting the authoritative SQL	13

1 Notation

- The **primary key** of each relation is its first attribute; all PKs are **Guid** (**uniqueidentifier**) unless noted. Junction relations use a *composite* PK.
- FK lines under each relation give the foreign key, its target, and its delete rule: **Cascade**, **Restrict** or **SetNull**.
- **(soft)** marks a *loose reference* — a **Guid** column that points at another relation but has *no* **FOREIGN KEY** constraint (integrity enforced in application code).
- ***UK** = unique index; **FUK** = filtered (partial) unique index; **enc** = encrypted column; **cents** = money stored as an integer.
- **{audit}** abbreviates the audit columns **CreatedAt**, **CreatedByUserId**, **UpdatedAt**, **UpdatedByUserId**, **RowVersion**; **{softdel}** abbreviates **IsDeleted**, **DeletedAt**, **DeletedByUserId**.

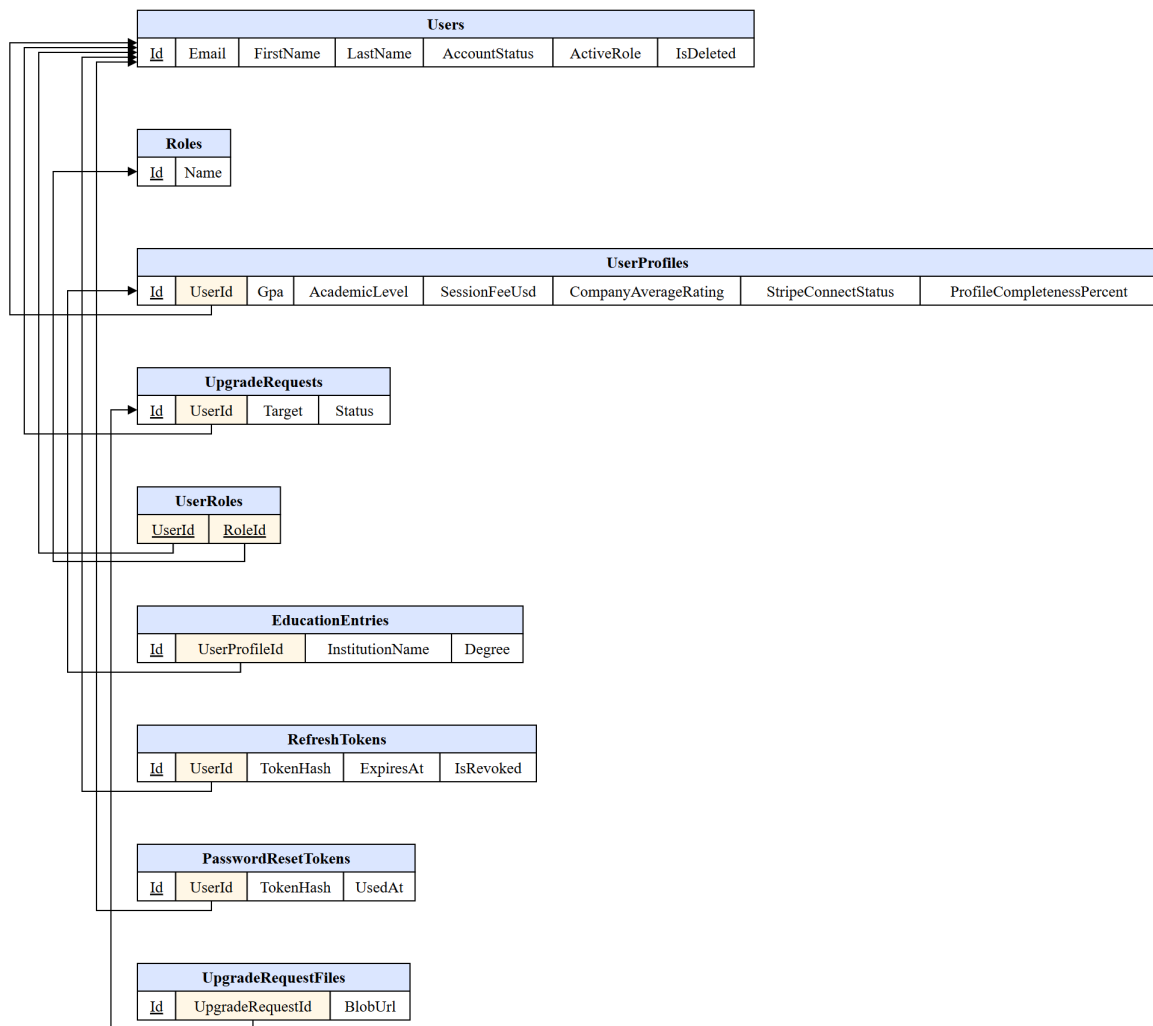
2 How the EER constructs were mapped

#	Construct	Rule applied	Example here
1	Regular entity	one relation; choose PK	USERS, SCHOLARSHIPS, BOOKINGS
2	Weak entity	relation incl. owner PK as part of FK	EDUCATION_ENTRIES, *_CHILDREN, *_FILES
3	1:1 relationship	FK (+ *UK) on the total-participation side	USERS—USER_PROFILES; BOOKINGS—CONSULTANT_REVIEWS
4	1:N relationship	FK on the N-side	SCHOLARSHIPS→APPLICATIONS
5	M:N relationship	new relation, composite PK of two FKs	USER_ROLES, FORUM_POST_TAGS
6	Multivalued attribute	separate relation <i>or</i> JSON column	*Json columns; SCHOLARSHIP_CHILDREN
7	N-ary relationship	relation with a FK to each participant	COMPANY_REVIEW_REQUESTS
8	Specialization (EER)	single relation + type discriminator	USER ISA {Student, Company, Consultant, Admin}
9	Union / category	—	not used

Deviation from step 6. Multivalued attributes (preferred countries, post tags, payout payment-ids) are stored as JSON string columns except where querying/ordering was needed (community tags became the real `FORUM_TAGS` + `FORUM_POST_TAGS` M:N). This denormalisation is recorded so the mapping matches the deployed schema.

3 Relational schema diagrams

Following Elmasri & Navathe’s relational-schema notation (Fig. 9.2), each relation is a box: the relation name sits in the header, and its attributes are drawn as a row of cells with the primary key underlined and foreign-key cells shaded. A **directed arrow** runs from every foreign-key cell to the exact primary-key cell it references, with the arrowhead resting on the referenced key. A **solid** arrow is a database-enforced FK; a **dashed** arrow (columns marked **soft**) is an application-enforced loose reference with no FK constraint. Arrows are routed orthogonally through a left-hand channel so that each one starts and ends on the correct cell without overlapping its neighbours.

**Figure 1:** Relational schema — Identity & Profile.

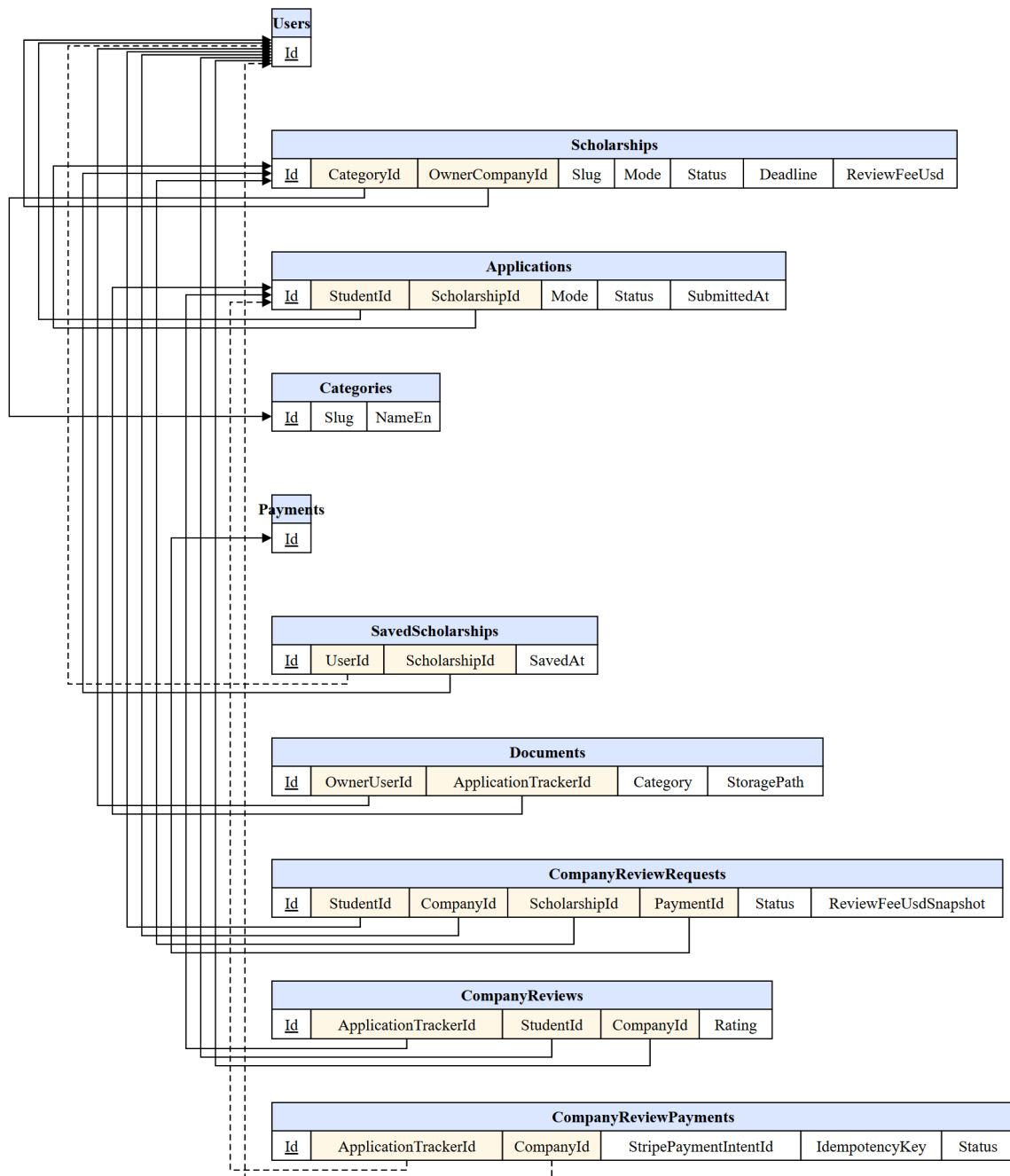


Figure 2: Relational schema — Scholarships, Applications & Documents.

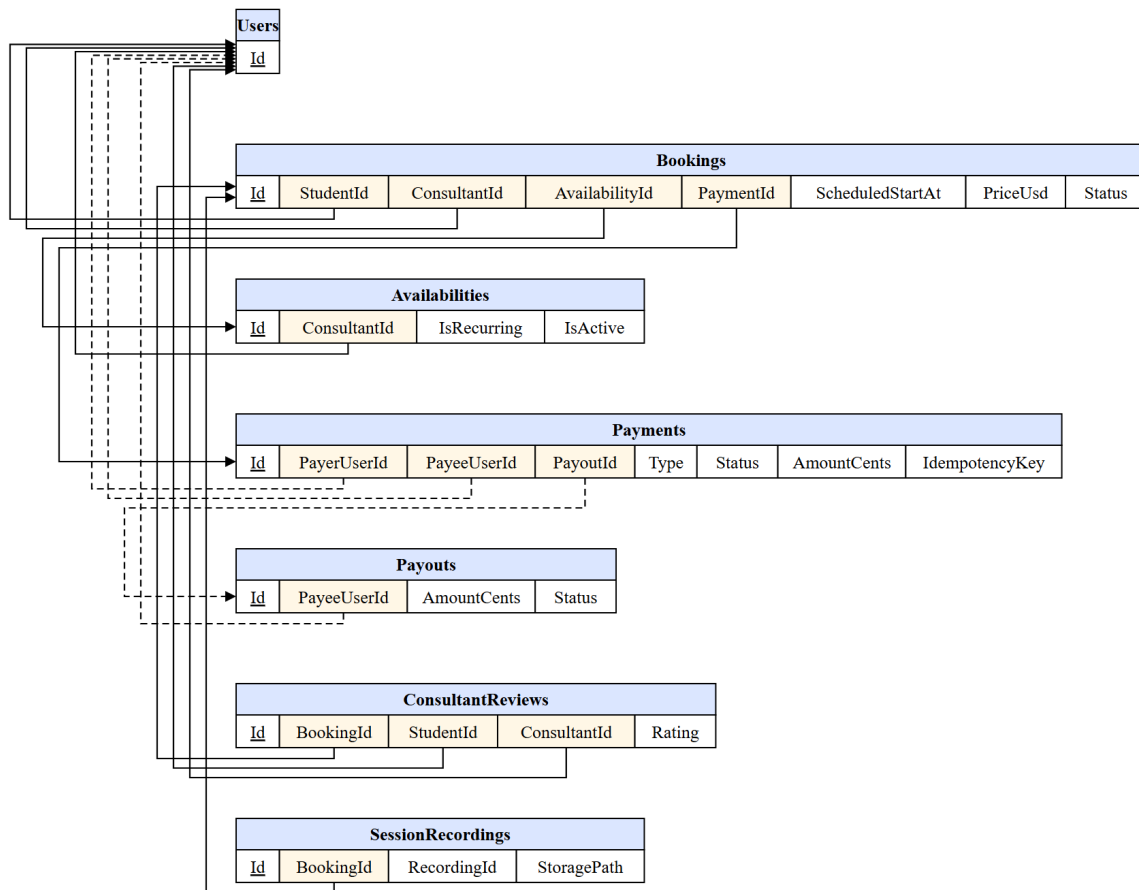
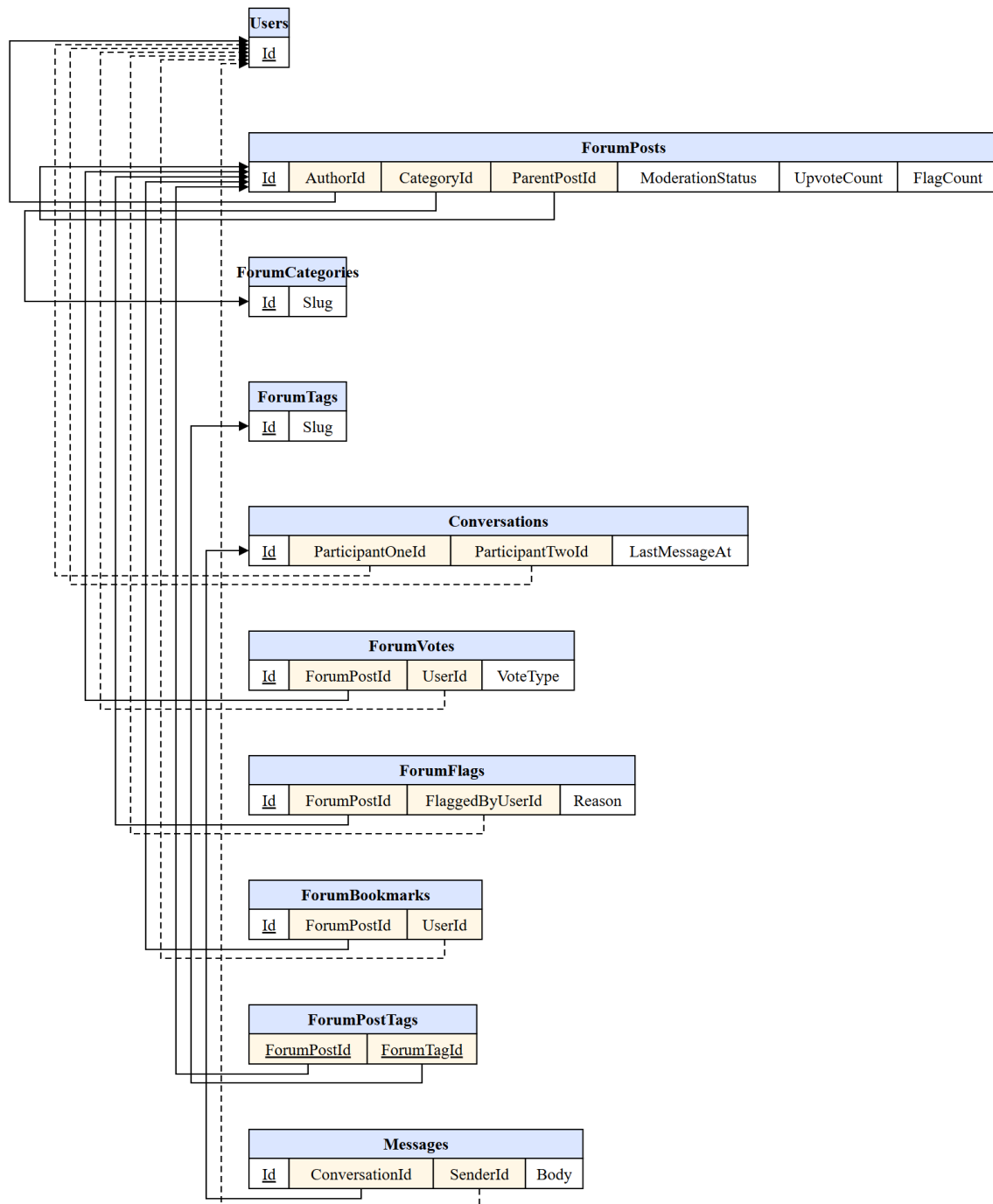


Figure 3: Relational schema — Booking, Payments & Ratings.

**Figure 4:** Relational schema — Community & Chat.

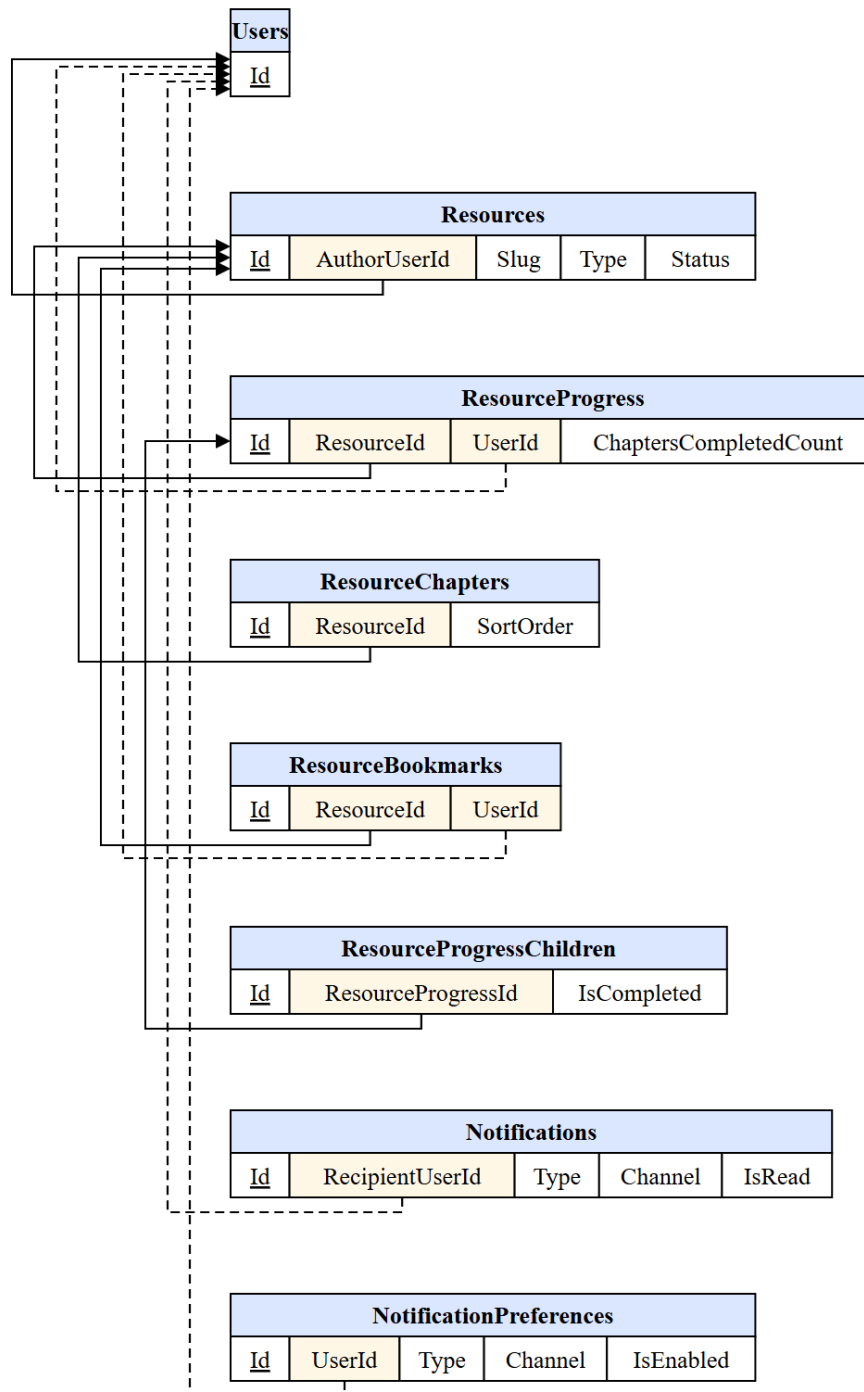


Figure 5: Relational schema — Resources Hub & Notifications.

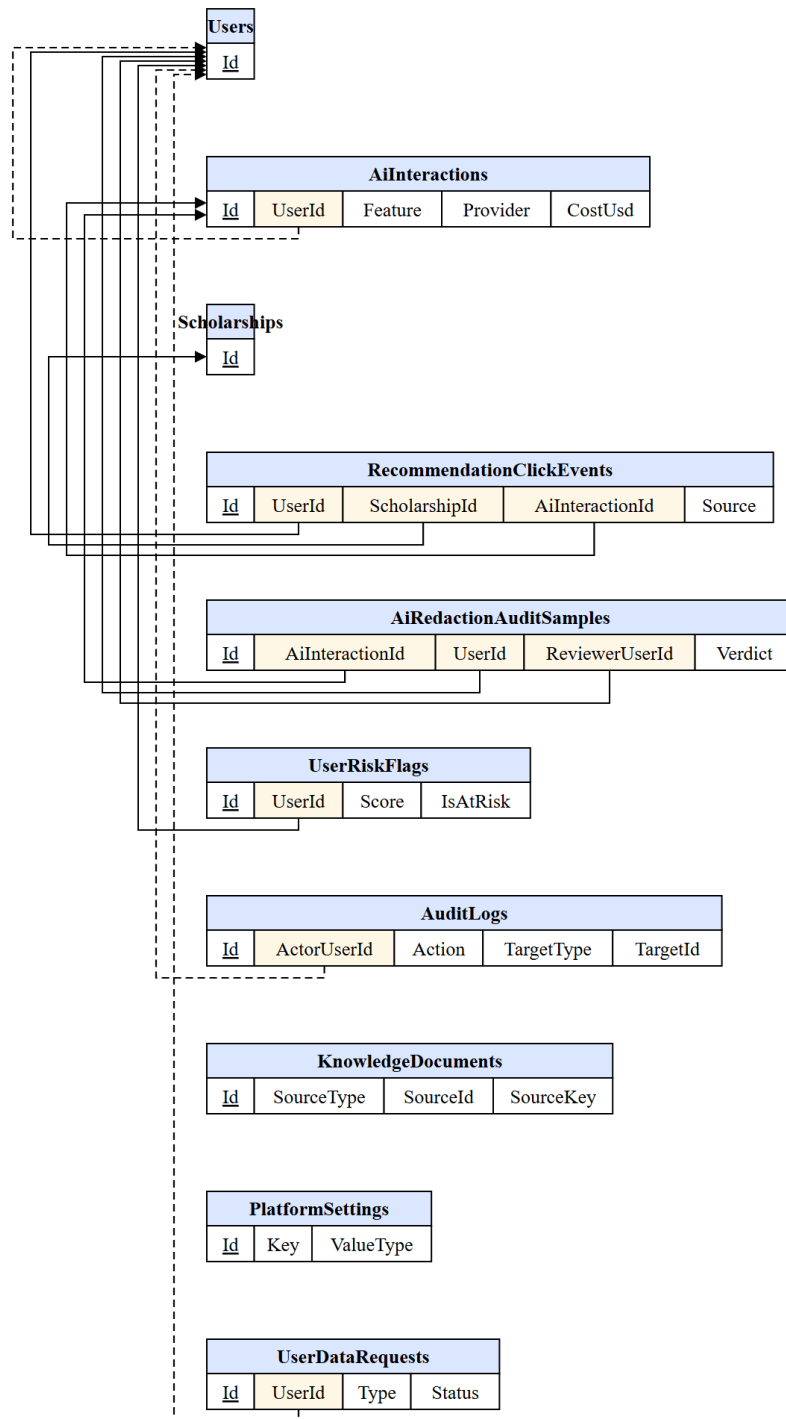


Figure 6: Relational schema — AI, Knowledge, Platform & Cross-cutting.

4 Relation schemas (textual)

4.1 Identity, Access & Profile

```
USERS( Id, Email*UK, NormalizedEmail*UK, FirstName, LastName, PasswordHash,
      SecurityStamp, EmailConfirmed, PhoneNumber, LockoutEnd, LockoutEnabled,
      AccessFailedCount, ProfileImageUrl, AccountStatus, ActiveRole,
      IsOnboardingComplete, PreferredLanguage, CountryOfResidence,
      LastLoginAt, {audit-thin}, {softdel} )
```

```

-- {audit-thin} = CreatedAt, UpdatedAt, RowVersion ONLY.  USERS does NOT carry
-- CreatedByUserId / UpdatedByUserId: ApplicationUser inherits IdentityUser<Guid>,
-- not AuditableEntity.  Standard Identity child tables: UserClaims, UserLogins,
-- UserTokens, RoleClaims.

ROLES( Id, Name, NormalizedName*UK, Description, CreatedAt )

USER_ROLES( UserId, RoleId )                -- M:N junction (step 5)
    PK = (UserId, RoleId)
    FK UserId -> USERS(Id) [Cascade];  FK RoleId -> ROLES(Id) [Cascade]
( UserClaims / UserLogins / UserTokens / RoleClaims : standard Identity tables )

USER_PROFILES( Id, UserId*UK, Biography(enc), DateOfBirth, Nationality,
    AcademicLevel, FieldOfStudy, Gpa, GpaScale, PreferredCountriesJson,
    OrganizationLegalName, OrganizationVerificationStatus, IsTaxRegistered,
    CompanyAverageRating, CompanyReviewCount, CompanyLowRatingFlaggedAt,
    SessionFeeUsd, SessionDurationMinutes, ExpertiseTagsJson, LanguagesJson,
    StripeConnectAccountId, StripeConnectStatus, ProfileCompletenessPercent,
    {audit} )
    FK UserId -> USERS(Id) [Cascade]        -- 1:1 (the *UK enforces it)
-- ~50 columns total: the extended onboarding set is elided here
-- (BiographyAr, LinkedInUrl, WebsiteUrl, Timezone, CurrentInstitution,
-- PreferredFieldsJson, Organization* x8, CompanyType/Description,
-- ContactPerson* x3, ProfessionalTitle, HighestDegree, FieldOfExpertise,
-- YearsOfExperience, PortfolioUrl, *VerifiedAt, *RejectedAt ...).
-- Filtered index IX_UserProfiles_CompanyLowRatingFlagged
-- WHERE [CompanyLowRatingFlaggedAt] IS NOT NULL                (R7)

EDUCATION_ENTRIES( Id, UserProfileId, InstitutionName, Degree, FieldOfStudy,
    StartDate, EndDate, Gpa, Description, {audit} )        -- weak (step 2)
    FK UserProfileId -> USER_PROFILES(Id) [Cascade]

EXPERTISE_TAGS( Id, Slug*UK, NameEn, NameAr, Category )    -- lookup, no FK

REFRESH_TOKENS( Id, UserId, TokenHash*UK, ExpiresAt, IsRevoked, RevokedAt,
    ReplacedByTokenId(soft), IpAddress, UserAgent, {audit} )
    FK UserId -> USERS(Id) [Cascade]

LOGIN_ATTEMPTS( Id, Email, UserId(soft), Succeeded, FailureReason, OccurredAt,
    IpAddress, UserAgent )

PASSWORD_RESET_TOKENS( Id, UserId, TokenHash*UK, ExpiresAt, UsedAt, {audit} )
    FK UserId -> USERS(Id) [Cascade]

UPGRADE_REQUESTS( Id, UserId(soft), Target, Status, Reason, ReviewerNotes,
    ReviewedByAdminId(soft), ReviewedAt, {audit}, {softdel} )

UPGRADE_REQUEST_FILES( Id, UpgradeRequestId, FileName, BlobUrl, SizeBytes,
    ContentType, UploadedAt )                            -- weak (step 2)
    FK UpgradeRequestId -> UPGRADE_REQUESTS(Id) [Cascade]

UPGRADE_REQUEST_LINKS( Id, UpgradeRequestId, Label, Url )  -- weak (step 2)
    FK UpgradeRequestId -> UPGRADE_REQUESTS(Id) [Cascade]

```

4.2 Scholarships, Applications & Documents

```

CATEGORIES( Id, Slug*UK, NameEn, NameAr, DescriptionEn, DescriptionAr,
    IconKey, DisplayOrder, {audit} )

SCHOLARSHIPS( Id, CategoryId, OwnerCompanyId, CreatedByAdminId(soft), Slug*UK,
    TitleEn, TitleAr, DescriptionEn, DescriptionAr, Mode,
    ExternalApplicationUrl, Status, Deadline, OpenedAt, ArchivedAt,
    IsFeatured, FeaturedOrder, FundingType, FundingAmountUsd, Currency,
    TargetLevel, TargetCountriesJson, FieldsOfStudyJson, TagsJson,

```

```

        ApplicationFormSchemaJson, RequiredDocumentsJson, ReviewFeeUsd,
        {audit}, {softdel} )
FK CategoryId      -> CATEGORIES(Id) [SetNull]
FK OwnerCompanyId -> USERS(Id)      [SetNull]    -- null = admin / external

SCHOLARSHIP_CHILDREN( Id, ScholarshipId(soft), ChildType, KeyEn, KeyAr,
        ValueEn, ValueAr, SortOrder )    -- weak EAV row; parent link is loose

SAVED_SCHOLARSHIPS( Id, UserId(soft), ScholarshipId(soft), SavedAt, Note )
*UK (UserId, ScholarshipId)              -- M:N bookmark; uniqueness only

APPLICATIONS( Id, StudentId, ScholarshipId, Mode, Status, FormDataJson,
        AttachedDocumentsJson, ExternalTrackingUrl, ExternalReferenceId,
        ExternalTitle, ExternalProvider, Deadline, SubmittedAt, WithdrawnAt,
        ReviewStartedAt, DecisionAt, DecisionReason, NextReminderAt,
        PersonalNotes(enc), {audit}, {softdel} )
FK StudentId      -> USERS(Id)          [Restrict]
FK ScholarshipId  -> SCHOLARSHIPS(Id) [Restrict]    -- null = external tracker
FUK (StudentId, ScholarshipId) WHERE ScholarshipId IS NOT NULL
        AND Status NOT IN ('Withdrawn','Rejected','Accepted') -- FR-057

APPLICATION_CHILDREN( Id, ApplicationTrackerId(soft), ChildType, Title, Content,
        MetadataJson, OccurredAt, ActorUserId(soft), SortOrder )

DOCUMENTS( Id, OwnerUserId, ApplicationTrackerId, FileName, ContentType,
        SizeBytes, StoragePath, Category, UploadedAt, {audit}, {softdel} )
FK OwnerUserId    -> USERS(Id)          [Restrict]
FK ApplicationTrackerId -> APPLICATIONS(Id) [SetNull]

COMPANY_REVIEW_REQUESTS( Id, StudentId, CompanyId, ScholarshipId,
        ApplicationTrackerId(soft), PaymentId, Status, ReviewFeeUsdSnapshot,
        Currency, SubmittedAt, AcceptedAt, RejectedAt, CompletedAt, ClosedAt,
        CancelledAt, ExpiredAt, PendingExpiresAt, CancelReason, RejectReason,
        {audit}, {softdel} )    -- N-ary (step 7)
-- Status enum = 11 values: Draft, Submitted, Pending, UnderReview,
-- Completed, Closed, Cancelled, Failed, CancelledByStudent,
-- RejectedByCompany, Expired. (R5)
FK StudentId      -> USERS(Id)          [Restrict]
FK CompanyId      -> USERS(Id)          [Restrict]
FK ScholarshipId  -> SCHOLARSHIPS(Id) [Restrict]
FK PaymentId      -> PAYMENTS(Id)       [SetNull]
FUK (StudentId, ScholarshipId)
        WHERE Status IN ('Draft','Submitted','Pending','UnderReview')

COMPANY_REVIEWS( Id, ApplicationTrackerId*UK, StudentId, CompanyId, Rating,
        Comment, IsHiddenByAdmin, AdminNote, {audit}, {softdel} ) -- 1:1
FK ApplicationTrackerId -> APPLICATIONS(Id) [Restrict] -- one per application
FK StudentId -> USERS(Id) [Restrict]; FK CompanyId -> USERS(Id) [Restrict]

-- legacy table, still in the schema (superseded by COMPANY_REVIEW_REQUESTS+PAYMENTS):
COMPANY_REVIEW_PAYMENTS( Id, ApplicationTrackerId, CompanyId, AmountUsd,
        ProfitShareAmountUsd, PayeeAmountUsd, StripePaymentIntentId*UK,
        IdempotencyKey*UK, Status, CapturedAt, RefundedAmountUsd,
        RefundReason, {audit} )    -- (R1)

```

4.3 Consultant Booking, Payments & Ratings

```

AVAILABILITIES( Id, ConsultantId, DayOfWeek, StartTime, EndTime,
        SpecificStartAt, SpecificEndAt, Timezone, IsRecurring, IsActive,
        {audit}, {softdel} )
FK ConsultantId -> USERS(Id) [Restrict]

BOOKINGS( Id, StudentId, ConsultantId, AvailabilityId, PaymentId,
        ScheduledStartAt, ScheduledEndAt, DurationMinutes, PriceUsd, StudentNotes,

```

```

MeetingRoomId, RecordingStartedAt, RecordingId,          -- PB-006
Status, RequestedAt, ConfirmedAt, RejectedAt,
ExpiredAt, CancelledAt, CompletedAt, CancellationReason,
CancelledByUserId(soft), StripePaymentIntentId, IsNoShowStudent,
IsNoShowConsultant, NoShowMarkedAt,
StudentJoinedAt, ConsultantJoinedAt,                    -- FR-217 attendance
{audit}, {softdel} )
FK StudentId      -> USERS(Id)          [Restrict]
FK ConsultantId   -> USERS(Id)          [Restrict]
FK AvailabilityId -> AVAILABILITIES(Id) [SetNull]
FK PaymentId      -> PAYMENTS(Id)       [SetNull]
FUK (ConsultantId, ScheduledStartAt) WHERE Status IN ('Requested','Confirmed')

PAYMENTS( Id, PayerUserId(soft), PayeeUserId(soft), RelatedBookingId(soft),
RelatedApplicationId(soft), PayoutId(soft), Type, Status,
AmountCents(cents), Currency, ProfitShareAmountCents(cents),
PayeeAmountCents(cents), RefundedAmountCents(cents), StripePaymentIntentId,
StripeChargeId, IdempotencyKey*UK, HeldAt, CapturedAt, RefundedAt,
RefundReason, FailureReason, {audit}, {softdel} )
-- NO database FKs by design (audit-decoupled)

PAYOUTS( Id, PayeeUserId(soft), AmountCents(cents), Currency, Status,
StripePayoutId, StripeConnectAccountId, InitiatedAt, PaidAt,
FailureReason, IncludedPaymentIdsJson, {audit} )

CONSULTANT_REVIEWS( Id, BookingId*UK, StudentId, ConsultantId, Rating, Comment,
IsHiddenByAdmin, AdminNote, {audit}, {softdel} )          -- 1:1
FK BookingId -> BOOKINGS(Id) [Restrict]          -- one review per booking
FK StudentId -> USERS(Id) [Restrict]; FK ConsultantId -> USERS(Id) [Restrict]

SESSION_RECORDINGS( Id, BookingId, RecordingId, StoragePath, ContentType,
SizeBytes, RecordedAt, {audit}, {softdel} )
FK BookingId -> BOOKINGS(Id) [Restrict]

STRIPE_WEBHOOK_EVENTS( Id, StripeEventId*UK, IsProcessed, ReceivedAt, ... )
PROFIT_SHARE_CONFIGS( Id, PaymentType, Percentage, EffectiveFrom, EffectiveTo,
SetByAdminId(soft), {audit} )          -- FUK one active per PaymentType
FINANCIAL_CONFIG_RULES( Id, PaymentType, FeeKind, ..., Status,
SetByAdminId(soft), {audit} )          -- FUK one active per PaymentType

```

4.4 Community & Chat

```

FORUM_CATEGORIES( Id, Slug*UK, NameEn, NameAr, DisplayOrder, IsActive, {audit} )

FORUM_POSTS( Id, AuthorId, CategoryId, ParentPostId, Title, BodyMarkdown,
ModerationStatus, UpvoteCount, DownvoteCount, FlagCount, ReplyCount,
IsAutoHidden, AutoHiddenAt, ModeratedByAdminId(soft), ModeratedAt,
ModerationNote, {audit}, {softdel} )
FK AuthorId      -> USERS(Id)          [Restrict]
FK CategoryId    -> FORUM_CATEGORIES(Id) [SetNull]
FK ParentPostId  -> FORUM_POSTS(Id)    [Restrict]    -- self; null = root

FORUM_POST_ATTACHMENTS( Id, ForumPostId, FileName, BlobUrl, ContentType, SizeBytes )
FK ForumPostId -> FORUM_POSTS(Id) [Cascade]
FORUM_VOTES( Id, ForumPostId, UserId(soft), VoteType, VotedAt )
FK ForumPostId -> FORUM_POSTS(Id) [Cascade]; *UK (ForumPostId, UserId)
FORUM_FLAGS( Id, ForumPostId, FlaggedByUserId(soft), Reason, FlaggedAt, IsValid )
FK ForumPostId -> FORUM_POSTS(Id) [Cascade]; *UK (ForumPostId, FlaggedByUserId)
FORUM_BOOKMARKS( Id, ForumPostId, UserId(soft), CreatedAt )
FK ForumPostId -> FORUM_POSTS(Id) [Cascade]; *UK (ForumPostId, UserId)
FORUM_TAGS( Id, Name, Slug*UK, CreatedAt )
FORUM_POST_TAGS( ForumPostId, ForumTagId )          -- M:N junction (step 5)
PK = (ForumPostId, ForumTagId)
FK ForumPostId -> FORUM_POSTS(Id) [Cascade]; FK ForumTagId -> FORUM_TAGS(Id) [Cascade]

```

```

CONVERSATIONS( Id, ParticipantOneId(soft), ParticipantTwoId(soft), LastMessageAt,
    LastMessageId(soft), IsArchivedForParticipantOne,
    IsArchivedForParticipantTwo, {audit} )
    *UK (ParticipantOneId, ParticipantTwoId)          -- one DM channel per pair
MESSAGES( Id, ConversationId, SenderId(soft), Body, SentAt, ReadAt,
    {audit}, {softdel} )
    FK ConversationId -> CONVERSATIONS(Id) [Cascade]
USER_BLOCKS( Id, BlockerId(soft), BlockedUserId(soft), BlockedAt, Reason )
    *UK (BlockerId, BlockedUserId)

```

4.5 Resources Hub & Notifications

```

RESOURCES( Id, AuthorUserId, Slug*UK, TitleEn, TitleAr, DescriptionEn,
    DescriptionAr, ContentMarkdownEn, ContentMarkdownAr, ExternalLinkUrl,
    CoverImageUrl, AuthorRole, Type, Status, CategorySlug, TagsJson,
    IsFeatured, FeaturedOrder, PublishedAt, ReviewedAt,
    ReviewedByAdminId(soft), RejectionReason, {audit}, {softdel} )
    FK AuthorUserId -> USERS(Id)
RESOURCE_CHAPTERS( Id, ResourceId, TitleEn, TitleAr, ContentMarkdownEn,
    ContentMarkdownAr, SortOrder, EstimatedReadMinutes )          -- weak
    FK ResourceId -> RESOURCES(Id) [Cascade]
RESOURCE_BOOKMARKS( Id, ResourceId, UserId(soft), BookmarkedAt )
    FK ResourceId -> RESOURCES(Id) [Cascade]; *UK (UserId, ResourceId)
RESOURCE_PROGRESS( Id, ResourceId, UserId(soft), ChaptersCompletedCount,
    LastAccessedAt, {audit} )
    FK ResourceId -> RESOURCES(Id) [Cascade]; *UK (UserId, ResourceId)
RESOURCE_PROGRESS_CHILDREN( Id, ResourceProgressId, ResourceChildId(soft),
    IsCompleted, CompletedAt )
    FK ResourceProgressId -> RESOURCE_PROGRESS(Id) [Cascade]
    *UK (ResourceProgressId, ResourceChildId)

NOTIFICATIONS( Id, RecipientUserId(soft), Type, Channel, TitleEn, TitleAr,
    BodyEn, BodyAr, DeepLink, MetadataJson, IsRead, ReadAt, Priority,
    IdempotencyKey, DispatchedAt, DispatchSucceeded, DispatchError,
    {audit}, {softdel} )
NOTIFICATION_PREFERENCES( Id, UserId(soft), Type, Channel, IsEnabled, {audit} )
    *UK (UserId, Type, Channel)

```

4.6 AI, Knowledge, Platform & Cross-cutting

```

AI_INTERACTIONS( Id, UserId(soft), Feature, Provider, ModelName, SessionId,
    PromptText, ResponseText, PromptTokens, CompletionTokens, CostUsd,
    MetadataJson, StartedAt, CompletedAt, ErrorMessage, {audit} )
RECOMMENDATION_CLICK_EVENTS( Id, UserId, ScholarshipId, AiInteractionId,
    ClickedAt, Source )
    FK UserId -> USERS(Id) [Restrict]; FK ScholarshipId -> SCHOLARSHIPS(Id) [Restrict]
    FK AiInteractionId -> AI_INTERACTIONS(Id) [SetNull]
AI_REDACTION_AUDIT_SAMPLES( Id, AiInteractionId*UK, UserId, ReviewerUserId,
    RedactedPrompt, SampledAt, Verdict, ReviewedAt )          -- 1:1
    FK AiInteractionId -> AI_INTERACTIONS(Id) [Restrict]
    FK UserId -> USERS(Id) [Restrict]; FK ReviewerUserId -> USERS(Id) [Restrict]
KNOWLEDGE_DOCUMENTS( Id, SourceType, SourceId(soft, polymorphic), SourceKey,
    TitleEn, TitleAr, ContentEn, ContentAr, ContentHash, Embedding,
    EmbeddingDimensions, EmbeddingModel, IndexedAt, MetadataJson, {audit} )
    *UK (SourceType, SourceKey)
PLATFORM_SETTINGS( Id, Key*UK, Value, ValueType, DescriptionEn, DescriptionAr,
    Category, UpdatedByAdminId(soft), {audit} )
AUDIT_LOGS( Id, ActorUserId(soft), Action, TargetType, TargetId(soft, polymorphic),
    BeforeJson, AfterJson, IpAddress, UserAgent, OccurredAt, CorrelationId,
    Summary )
USER_DATA_REQUESTS( Id, UserId(soft), Type, Status, RequestedAt,
    ScheduledProcessAt, CompletedAt, CancelledAt, DownloadUrl,
    DownloadExpiresAt, FailureReason, {audit} )

```

```

SUCCESS_STORIES( Id, StudentId(soft, nullable), AuthorDisplayName, AuthorImageUrl,
  HeadlineEn, HeadlineAr, BodyEn, BodyAr, ScholarshipNameEn, ScholarshipNameAr,
  CountryCode, IsApproved, IsFeatured, FeaturedOrder, {audit}, {softdel} )
USER_RISK_FLAGS( Id, UserId*UK, Score, IsAtRisk, Reason, ComputedAt,
  SourceRefreshId(soft) ) -- 1:1
FK UserId -> USERS(Id) [Cascade] -- one flag row per user

```

5 Referential-integrity summary

Delete rule	Used for	Why
Cascade	profile, education, tokens, upgrade files/links, forum attachments / votes / flags / bookmarks / post-tags, messages, resource chapters / bookmarks / progress, user-risk-flag	child has no meaning without its parent
Restrict	applications, documents, bookings, all reviews, recordings, recommendation clicks, redaction samples, every FK into USERS	protect history; also avoids SQL Server multiple-cascade-paths (error 1785)
SetNull	scholarship→category / owner, document→application, booking→availability / payment, request→payment, post→category	keep the row, orphan the optional link
none (loose)	payments, payouts, notifications, votes / flags / bookmarks' user, chat participants / sender, audit actor + target, AI userId, knowledge sourceId, settings / requests / stories user	decouple high-volume / audit / analytics rows so deletes & anonymisation never cascade-break

6 Exporting the authoritative SQL

The relation schemas above can be regenerated as real DDL straight from the EF Core migrations:

```

cd server
dotnet ef migrations script ~
--project src/ScholarPath.Infrastructure ~
--startup-project src/ScholarPath.API > ../docs/diagrams/schema.sql

```